



Exercise Problem 5 (EP_05)

Subject: *ENE/EIE 334 Digital System Engineering*

Series : *Computation Series*

Program: *Bachelor of Engineering in ECEE¹, Bachelor of Engineering in EIE²*

Semester: 1/2026

Topic: *Arithmetic and Logic Unit (ALU): Arith Module: Binary Adder (binadder)*

Designer: *Pinit Kumhom*

Revision History:

Last Updated: September, 2026 for Semester 1/2026

First Released: September, 2026 for Semester 1/2026

Objectives

1. To learn about tradeoff in designing bitwise boolean module for Beta ALU targeting CMOS technology
2. To practice using simulation tool particularly JSIM in verification and evaluation of binary adder modules.

Name-Surname (ชื่อ-สกุล) ธนิตา อธิราชบุรี

Student ID (รหัสประจำ นศ.) 67070502472

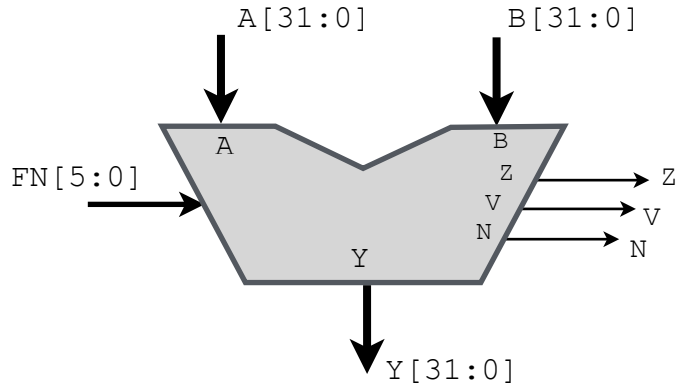
¹ **ECEE** stands for **Electrical Communication and Electronic Engineering**, a curriculum's name in the department of electronic and telecommunication, faculty of engineering, KMUTT

² **EIE** stands for **Electronic and Infocommunication Engineering (EIE)**, a curriculum's name in the department of electronic and telecommunication, faculty of engineering, KMUTT

ALU Module for Beta CPU

Interface Specification

Port Name	Dir	Description
A[31:0]	In	32-bit operand 1
B[31:0]	In	32-bit operand 2
Y[31:0]	Out	32-bit result
FN[5:0]	In	6-bit function selection
Z	Out	zero flag (1 bit)
V	Out	overflow flag (1 bit)
N	Out	negative flag (1 bit)



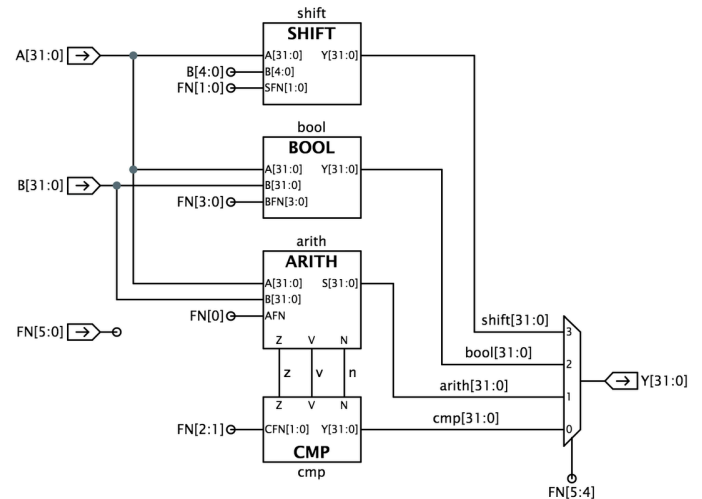
Functional Specification

FN[5:0]	Operation	Output value Y[31:0]
00-011	CMPEQ	$Y = (A == B)$
00-101	CMPLT	$Y = (A < B)$
00-111	CMPLT	$Y = (A \leq B)$
01---0	32-bit ADD	$Y = A + B$
01---1	32-bit SUBTRACT	$Y = A - B$
10abcd	Bit-wise Boolean	$Y[i] = F_{abcd}(A[i], B[i])$
11--00	Logical Shift left (SHL)	$Y = A \ll B$
11--01	Logical Shift right (SHR)	$Y = A \gg B$
11--11	Arithmetic Shift right (SRA)	$Y = A \gg B$ (sign extended)

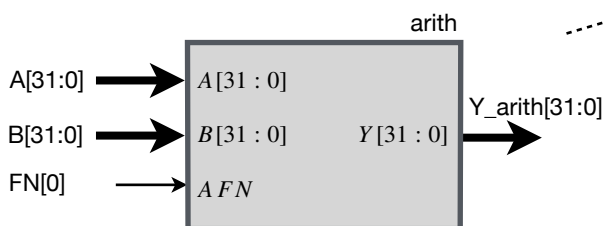
```

Z <= '1' when Y=0 else
    '0';
N <= '1' when Y<0 else
    '0';
V <= '1' when Y<(-2^31) or Y>(2^31-1) else
    '0';
    
```

Structural (Architectural) Design

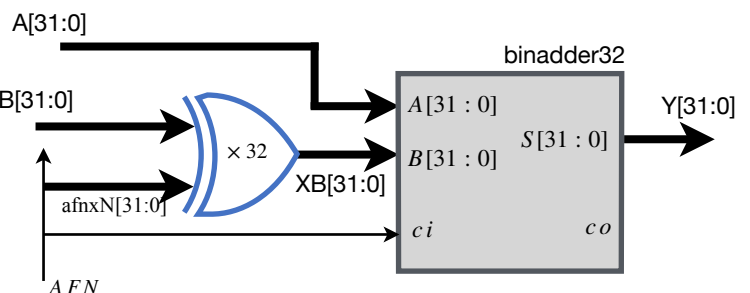


ARITH Module's Specification



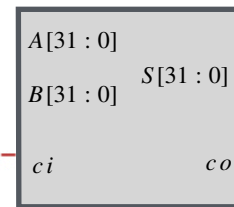
$Y \leq A - B$ when $afn = 1$ else $A + B$

ARITH module Design



Modeling the binadder32 module Using VHDL

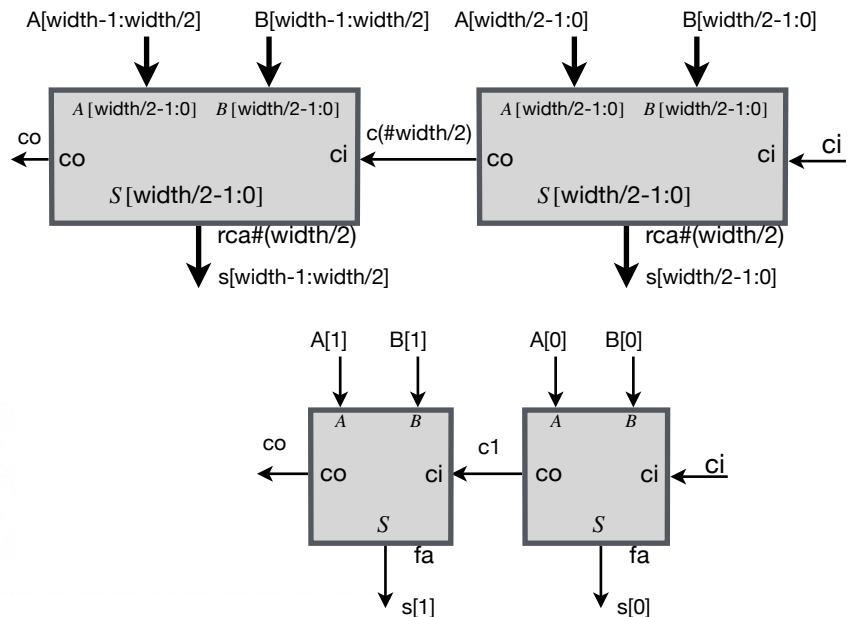
binadder32



Design Concept#1: Ripple-Carry Adder (RCA)

```
-- fa module
-- Description:
-- full 1-bit binary adder
```

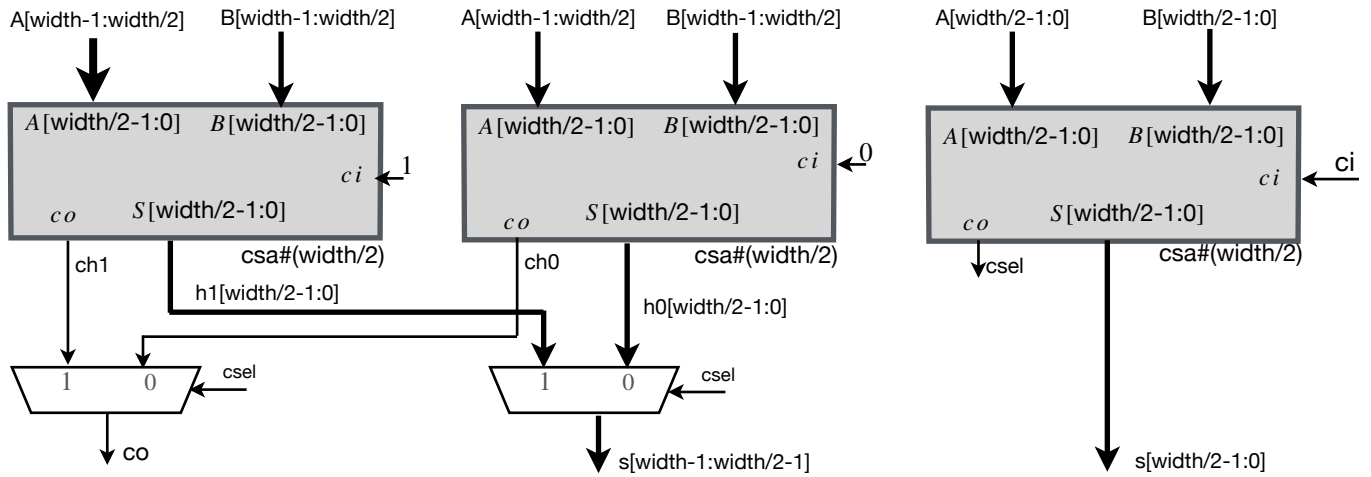
```
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity fa is
    port (
        a, b : in std_logic;
        ci : in std_logic;
        co, s : out std_logic);
end entity fa;
architecture stdcell of fa is
    signal n1, n2, n3 : std_logic;
begin -- architecture beh
    n1 <= nand2(a, b) after tpd_nand2;
    n2 <= xor2(a, b) after tpd_xor2;
    n3 <= nand2(n2, ci) after tpd_nand2;
    s <= xor2(n2, ci) after tpd_xor2;
    co <= nand2(n1, n3) after tpd_nand2;
end architecture stdcell;
```



```
-- rca module
-- Description: a parameterizable ripple-carry binary adder
```

```
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity rca is
    generic (
        width : natural);
    port(a, b : in std_logic_vector(width-1 downto 0);
        ci : in std_logic;
        co : out std_logic;
        s : out std_logic_vector(width-1 downto 0));
end entity rca;
architecture stdcell of rca is
    -- constant width : natural := 32;
    signal c : std_logic_vector(width downto 0);
begin -- architecture rca_stdcell
    fa_gen: for i in 0 to width-1 generate
        fa_i: entity work.fa(stdcell)
            port map (
                a => a(i),
                b => b(i),
                ci => c(i),
                s => s(i),
                co => c(i+1));
    end generate fa_gen;
    -- carry in
    c(0) <= ci;
    -- carry out
    co <= c(width);
end architecture stdcell;
```

Design Concept#2: Carry-Select Adder (CSA)



-- 2-bit Carry-select Binary Adder (CSA2)

-- Description:

-- an 2-bit carry-select unsigned adder targeting JSIM standard cells

```
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity csa2 is
    port (
        a, b : in  std_logic_vector(1 downto 0);
        ci : in  std_logic;
        co : out std_logic;
        s : out std_logic_vector(1 downto 0));
end entity csa2;
architecture stdcell of csa2 is
    constant width : natural := 2;
    signal h0, h1 : std_logic;
    signal csel, ch0, ch1 : std_logic;
begin -- architecture cla_stdcell
    adder_L: entity work.fa_csa(stdcell)
        port map (
            a => a(0),
            b => b(0),
            ci => ci,
            co => csel,
            s => s(0));
    adder_H0: entity work.fa_csa(stdcell)
        port map (
            a => a(1),
            b => b(1),
            ci => '0',
            co => ch0,
            s => h0);
    adder_H1: entity work.fa_csa(stdcell)
        port map (
            a => a(1),
            b => b(1),
            ci => '1',
            co => ch1,
            s => h1);
    -- selecting correct sum of the high bits
    hbits_sel: entity stdcell_lib.mux2(beh)
        port map (s => csel, d0 => h0, d1 => h1, z => s(1));
    -- selecting correct MSB carry out (co)
    ch_sel: entity stdcell_lib.mux2(beh)
        port map (s => csel, d0 => ch0, d1 => ch1, z => co);
end architecture stdcell;
```

```
-- fa_csa module
-- Description:
-- full 1-bit binary adder

library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity fa_csa is
    port (
        a, b : in  std_logic;
        ci : in  std_logic;
        co : out std_logic;
        s : out std_logic);
end entity fa_csa;
architecture stdcell of fa_csa is
    signal n1, n2, n3, n4, n5 : std_logic;
begin -- architecture beh
    n1 <= nand2(a, b)      after tpd_nand2;
    n2 <= xor2(a, b)      after tpd_xor2;
    s <= xor2(n2, ci)     after tpd_xor2;
    n3 <= nand2(ci, n2)   after tpd_nand2;
    co <= nand2(n1, n3)   after tpd_nand2;
end architecture stdcell;
```

```

-----
-- 4-bit Carry-select Binary Adder (CSA4)
-----

-- Description:
--   an 4-bit carry-select unsigned adder targeting JSIM standard cells

library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity csa4 is

    port (
        a, b : in  std_logic_vector(3 downto 0);
        ci   : in  std_logic;
        co   : out std_logic;
        s    : out std_logic_vector(3 downto 0));

end entity csa4;
architecture stdcell of csa4 is
    constant width : natural := 4;
    signal h0, h1 : std_logic_vector(width/2-1 downto 0);
    signal csel, ch0, ch1 : std_logic;
begin -- architecture cla_stdcell
    adder_L: entity work.csa2(stdcell)
        port map (
            a => a(width/2-1 downto 0),
            b => b(width/2-1 downto 0),
            ci => ci,
            co => csel,
            s => s(width/2-1 downto 0));
    adder_H0: entity work.csa2(stdcell)
        port map (
            a => a(width-1 downto width/2),
            b => b(width-1 downto width/2),
            ci => '0',
            co => ch0,
            s => h0(width/2-1 downto 0));
    adder_H1: entity work.csa2(stdcell)
        port map (
            a => a(width-1 downto width/2),
            b => b(width-1 downto width/2),
            ci => '1',
            co => ch1,
            s => h1(width/2-1 downto 0));
    -- selecting correct sum of the high bits using mux2xN
    hbits_sel: entity stdcell_lib.mux2xN(beh)
        generic map (n => width/2)
        port map (s => csel, d0 => h0, d1 => h1, z => s(width-1 downto width/2))
    -- selecting correct MSB carry out (co)
    ch_sel: entity stdcell_lib.mux2(beh)
        port map (s => csel, d0 => ch0, d1 => ch1, z => co);
end architecture stdcell;

```

```

-----
-- 8-bit Carry-select Binary Adder (CSA8)
-----

-- Description:
--   an 8-bit carry-select unsigned adder targeting JSIM standard cells

library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity csa8 is

    port (
        a, b : in  std_logic_vector(7 downto 0);
        ci   : in  std_logic;
        co   : out std_logic;
        s    : out std_logic_vector(7 downto 0));

end entity csa8;
architecture stdcell of csa8 is
    constant width : natural := 8;
    signal h0, h1 : std_logic_vector(width/2-1 downto 0);
    signal csel, ch0, ch1 : std_logic;
begin -- architecture stdcell
    adder_L: entity work.csa4(stdcell)
        port map (
            a => a(width/2-1 downto 0),
            b => b(width/2-1 downto 0),
            ci => ci,
            co => csel,
            s => s(width/2-1 downto 0));
    adder_H0: entity work.csa4(stdcell)
        port map (
            a => a(width-1 downto width/2),
            b => b(width-1 downto width/2),
            ci => '0',
            co => ch0,
            s => h0(width/2-1 downto 0));
    adder_H1: entity work.csa4(stdcell)
        port map (
            a => a(width-1 downto width/2),
            b => b(width-1 downto width/2),
            ci => '1',
            co => ch1,
            s => h1(width/2-1 downto 0));
    -- selecting correct sum of the high bits using mux2xN
    hbits_sel: entity stdcell_lib.mux2xN(beh)
        generic map (n => width/2)
        port map (s => csel, d0 => h0, d1 => h1, z => s(width-1 downto width/2))
    -- selecting correct MSB carry out (co)
    ch_sel: entity stdcell_lib.mux2(beh)
        port map (s => csel, d0 => ch0, d1 => ch1, z => co);
end architecture stdcell;

```

```

-- 16-bit Carry-select Binary Adder (CSA16)
--
-- Description:
--   an 16-bit carry-select unsigned adder targeting JSIM standard cells
--
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity csa16 is

    port (
        a, b : in  std_logic_vector(15 downto 0);
        ci   : in  std_logic;
        co   : out std_logic;
        s    : out std_logic_vector(15 downto 0));

end entity csa16;
architecture stdcell of csa16 is
    constant width : natural := 16;
    signal h0, h1 : std_logic_vector(width/2-1 downto 0);
    signal csel, ch0, ch1 : std_logic;
begin -- architecture stdcell
    adder_L: entity work.csa8(stdcell)
        port map (
            a => a(width/2-1 downto 0),
            b => b(width/2-1 downto 0),
            ci => ci,
            co => csel,
            s => s(width/2-1 downto 0));
    adder_H0: entity work.csa8(stdcell)
        port map (
            a => a(width-1 downto width/2),
            b => b(width-1 downto width/2),
            ci => '0',
            co => ch0,
            s => h0(width/2-1 downto 0));
    adder_H1: entity work.csa8(stdcell)
        port map (
            a => a(width-1 downto width/2),
            b => b(width-1 downto width/2),
            ci => '1',
            co => ch1,
            s => h1(width/2-1 downto 0));
    -- selecting correct sum of the high bits using mux2xN
    hbits_sel: entity stdcell_lib.mux2xN(beh)
        generic map (n => width/2)
        port map (s => csel, d0 => h0, d1 => h1, z => s(width-1 downto width/2));
    -- selecting correct MSB carry out (co)
    ch_sel: entity stdcell_lib.mux2(beh)
        port map (s => csel, d0 => ch0, d1 => ch1, z => co);
end architecture stdcell;

-- 32-bit Carry-select Binary Adder (CSA32)
--
-- Description:
--   an 32-bit carry-select unsigned adder targeting JSIM standard cells
--
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity csa32 is

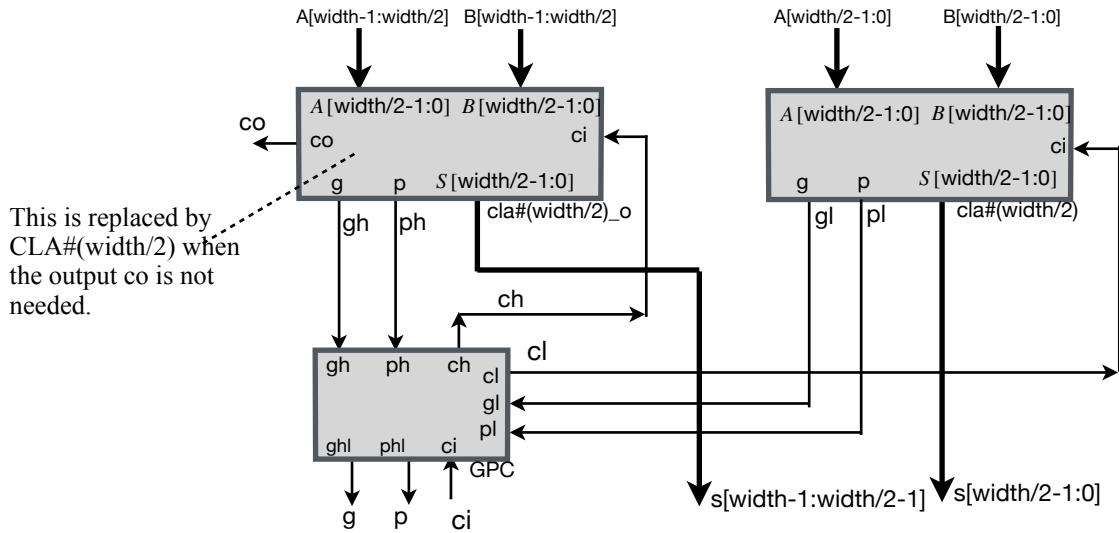
    port (
        a, b : in  std_logic_vector(31 downto 0);
        ci   : in  std_logic;
        co   : out std_logic;
        s    : out std_logic_vector(31 downto 0));

end entity csa32;
architecture stdcell of csa32 is
    constant width : natural := 32;
    signal h0, h1 : std_logic_vector(width/2-1 downto 0);
    signal csel, ch0, ch1 : std_logic;
begin -- architecture stdcell
    adder_L: entity work.csa16(stdcell)
        port map (
            a => a(width/2-1 downto 0),
            b => b(width/2-1 downto 0),
            ci => ci,
            co => csel,
            s => s(width/2-1 downto 0));
    adder_H0: entity work.csa16(stdcell)
        port map (
            a => a(width-1 downto width/2),
            b => b(width-1 downto width/2),
            ci => '0',
            co => ch0,
            s => h0(width/2-1 downto 0));
    adder_H1: entity work.csa16(stdcell)
        port map (
            a => a(width-1 downto width/2),
            b => b(width-1 downto width/2),
            ci => '1',
            co => ch1,
            s => h1(width/2-1 downto 0));
    -- selecting correct sum of the high bits using mux2xN
    hbits_sel: entity stdcell_lib.mux2xN(beh)
        generic map (n => width/2)
        port map (s => csel, d0 => h0, d1 => h1, z => s(width-1 downto width/2));
    -- selecting correct MSB carry out (co)
    ch_sel: entity stdcell_lib.mux2(beh)
        port map (s => csel, d0 => ch0, d1 => ch1, z => co);
end architecture stdcell;

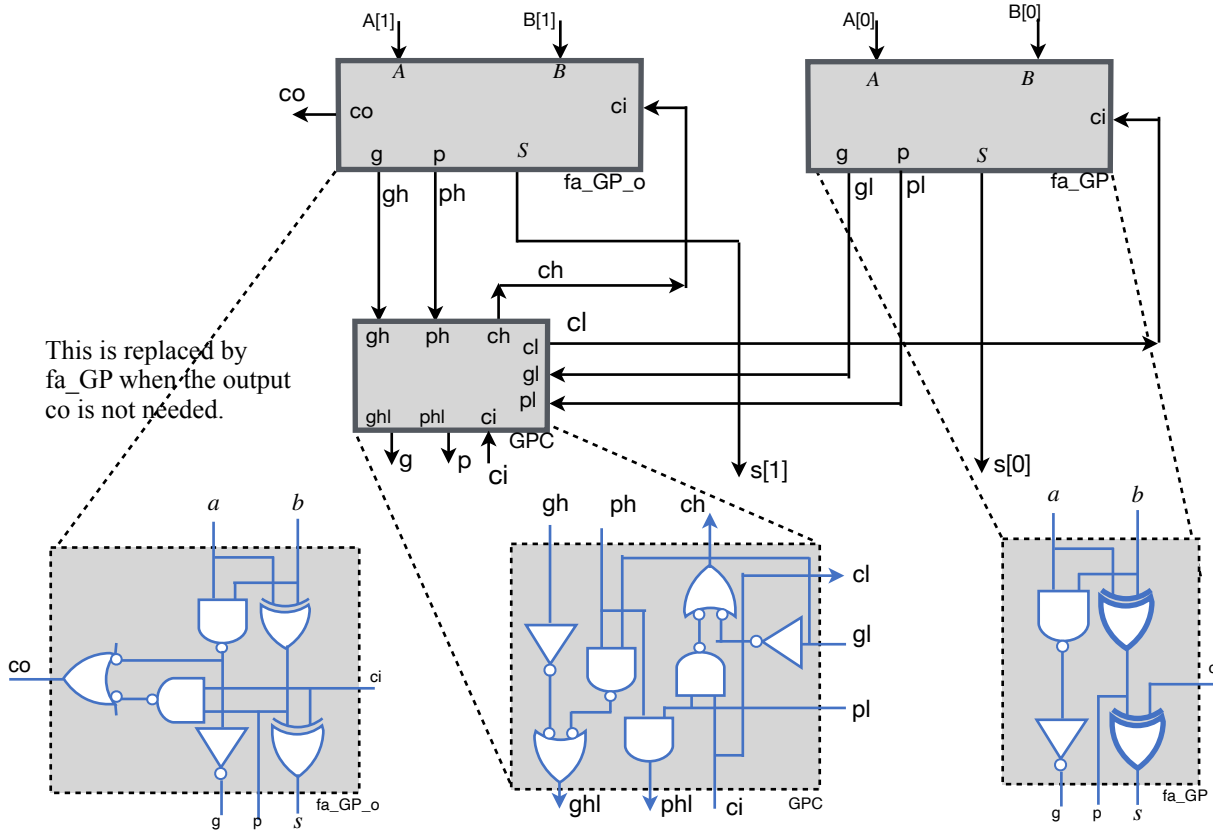
```

Design Concept#3: Carry-Lookahead Adder (CLA)

Generic structure of CLA#width composed of 2 CLA#(width/2) and GPC module



Structure of CLA2_o composing of fa_GP (=CLA1), fa_GP_co(=CLA1_o) and GPC module



```

-- fa_gp_co module
-- Description:
-- full 1-bit adder for MSB of the carry-look ahead add
-- which also produce the carry-out output in addition
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity fa_gp_co is
port (
    a, b : in std_logic;
    ci : in std_logic;
    co : out std_logic;
    s : out std_logic;
    g, p : out std_logic);
end entity fa_gp_co;
architecture stdcell of fa_gp_co is
    signal n1, n2, n3, n4, n5 : std_logic;
begin
    -- architecture beh
    n1 <= nand2(a, b) after tpd_nand2;
    n2 <= xor2(a, b) after tpd_xor2;
    s <= xor2(n2, ci) after tpd_xor2;
    g <= inv(n1) after tpd_inv;
    p <= n2;
    n3 <= nand2(ci, n2) after tpd_nand2;
    co <= nand2(n1, n3) after tpd_nand2;
end architecture stdcell;

```

```

-- gpc module
-- combining gh, ph, gl, pl to generate g, p in the higher level(ghl, phl) and
-- co gl, pl and ci from higher level to generate 2 carries (ch, cl)
-- of lower level of the tree structure (counting the level from the leaves)
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity gpc is
port (
    gh, ph : in std_logic;
    gl, pl : in std_logic;
    ci : in std_logic;
    ch, cl : out std_logic;
    ghl, phl : out std_logic);
end entity gpc;
architecture stdcell of gpc is
    signal n1, n2, n3, n4, n5 : std_logic;
begin
    -- architecture beh
    n1 <= nand2(ph, gl) after tpd_nand2;
    n2 <= inv(gh) after tpd_inv;
    ghl <= nand2(n1, n2) after tpd_nand2;
    n3 <= nand2(ph, pl) after tpd_nand2;
    phl <= inv(n3) after tpd_inv;
    cl <= ci;
    n4 <= nand2(pl, ci) after tpd_nand2;
    n5 <= inv(gl) after tpd_inv;
    ch <= nand2(n4, n5) after tpd_nand2;
end architecture stdcell;

```

```

-- fa_gp module
-- Description:
-- full 1-bit adder for carry-look ahead add (cla)
-- which does not require carry-out output but must generate G and P output
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity fa_gp is
port (
    a, b : in std_logic;
    ci : in std_logic;
    s : out std_logic;
    g, p : out std_logic);
end entity fa_gp;
architecture stdcell of fa_gp is
    signal n1, n2, n3, n4, n5 : std_logic;
begin
    -- architecture beh
    n1 <= nand2(a, b) after tpd_nand2;
    n2 <= xor2(a, b) after tpd_xor2;
    s <= xor2(n2, ci) after tpd_xor2;
    g <= inv(n1) after tpd_inv;
    p <= n2;
end architecture stdcell;

```

-- 2-bit Carry-lookahead Binary Adder (CLA2, CLA2_co)

-- Description:
-- an 2-bit carry-lookahead unsigned adder targeting JSIM standard cells

-- cla2: cla2 without co from MSB

```
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity cla2 is
    port (
        a, b : in  std_logic_vector(1 downto 0);
        ci : in  std_logic;
        s : out std_logic_vector(1 downto 0);
        g, p : out std_logic);
end entity cla2;
architecture stdcell of cla2 is
    signal gh, ph, gl, pl : std_logic;
    signal ch, cl : std_logic;
begin -- architecture cla_stdcell
    adder_L: entity work.fa_gp(stdcell)
        port map (
            a => a(0),
            b => b(0),
            ci => ci,
            s => s(0),
            g => gl,
            p => pl);
    adder_H: entity work.fa_gp(stdcell)
        port map (
            a => a(1),
            b => b(1),
            ci => ch,
            s => s(1),
            g => gh,
            p => ph);
    -- generating ch, cl, g, h using gpc
    gpc_0: entity work.gpc(stdcell)
        port map (
            gh => gh, ph => ph,
            gl => gl, pl => pl,
            ci => ci,
            ghl => g, phl => p,
            ch => ch, cl => cl
        );
end architecture stdcell;
```

-- cla2_co: cla2 with co from MSB

```
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity cla2_co is
    port (
        a, b : in  std_logic_vector(1 downto 0);
        ci : in  std_logic;
        co : out std_logic;
        s : out std_logic_vector(1 downto 0);
        g, p : out std_logic);
end entity cla2_co;
architecture stdcell of cla2_co is
    signal gh, ph, gl, pl : std_logic;
    signal ch, cl : std_logic;
begin -- architecture cla_stdcell
    adder_L: entity work.fa_gp(stdcell)
        port map (
            a => a(0),
            b => b(0),
            ci => ci,
            s => s(0),
            g => gl,
            p => pl);
    adder_H: entity work.fa_gp_co(stdcell)
        port map (
            a => a(1),
            b => b(1),
            ci => ch,
            co => co,
            s => s(1),
            g => gh,
            p => ph);
    -- generating ch, cl, g, h using gpc
    gpc_0: entity work.gpc(stdcell)
        port map (
            gh => gh, ph => ph,
            gl => gl, pl => pl,
            ci => ci,
            ghl => g, phl => p,
            ch => ch, cl => cl
        );
end architecture stdcell;
```

-- 4-bit Carry-lookahead Binary Adder

-- Description:
-- an 4-bit carry-lookahead unsigned adder targeting JSIM standard cells

-- cla4: cla4 without co from MSB

```
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity cla4 is
    port (
        a, b : in  std_logic_vector(3 downto 0);
        ci : in  std_logic;
        s : out std_logic_vector(3 downto 0);
        g, p : out std_logic);
end entity cla4;
architecture stdcell of cla4 is
    signal gh, ph, gl, pl : std_logic;
    signal ch, cl : std_logic;
begin -- architecture cla_stdcell
    adder_L: entity work.cla2(stdcell)
        port map (
            a => a(1 downto 0),
            b => b(1 downto 0),
            ci => ci,
            s => s(1 downto 0),
            g => gl,
            p => pl);
    adder_H: entity work.cla2(stdcell)
        port map (
            a => a(3 downto 2),
            b => b(3 downto 2),
            ci => ch,
            s => s(3 downto 2),
            g => gh,
            p => ph);
    -- generating ch, cl, g, h using gpc
    gpc_0: entity work.gpc(stdcell)
        port map (
            gh => gh, ph => ph,
            gl => gl, pl => pl,
            ci => ci,
            ghl => g, phl => p,
            ch => ch, cl => cl
        );
end architecture stdcell;
```

-- cla4_co: cla4 with co from MSB

```
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity cla4_co is
    port (
        a, b : in  std_logic_vector(3 downto 0);
        ci : in  std_logic;
        co : out std_logic;
        s : out std_logic_vector(3 downto 0);
        g, p : out std_logic);
end entity cla4_co;
architecture stdcell of cla4_co is
    signal gh, ph, gl, pl : std_logic;
    signal ch, cl : std_logic;
begin -- architecture cla_stdcell
    adder_L: entity work.cla2(stdcell)
        port map (
            a => a(1 downto 0),
            b => b(1 downto 0),
            ci => ci,
            s => s(1 downto 0),
            g => gl,
            p => pl);
    adder_H: entity work.cla2_co(stdcell)
        port map (
            a => a(3 downto 2),
            b => b(3 downto 2),
            ci => ch,
            co => co,
            s => s(3 downto 2),
            g => gh,
            p => ph);
    -- generating ch, cl, g, h using gpc
    gpc_0: entity work.gpc(stdcell)
        port map (
            gh => gh, ph => ph,
            gl => gl, pl => pl,
            ci => ci,
            ghl => g, phl => p,
            ch => ch, cl => cl
        );
end architecture stdcell;
```

-- 8-bit Carry-lookahead Binary Adder

-- Description:
-- an 8-bit carry-lookahead unsigned adder targeting JSIM standard cells

-- cla8: cla8 without co from MSB

```
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity cla8 is

    port (
        a, b : in  std_logic_vector(7 downto 0);
        ci : in  std_logic;
        s : out std_logic_vector(7 downto 0);
        g, p : out std_logic);

end entity cla8;
architecture stdcell of cla8 is
    signal gh, ph, gl, pl : std_logic;
    signal ch, cl : std_logic;
begin -- architecture cla_stdcell
    adder_L: entity work.cla4(stdcell)
        port map (
            a => a(3 downto 0),
            b => b(3 downto 0),
            ci => ci,
            s => s(3 downto 0),
            g => gl,
            p => pl);
    adder_H: entity work.cla4(stdcell)
        port map (
            a => a(7 downto 4),
            b => b(7 downto 4),
            ci => ch,
            s => s(7 downto 4),
            g => gh,
            p => ph);
    -- generating ch, cl, g, h using gpc
    gpc_0: entity work.gpc(stdcell)
        port map (
            gh => gh, ph => ph,
            gl => gl, pl => pl,
            ci => ci,
            ghl => g, phl => p,
            ch => ch, cl => cl
        );
end architecture stdcell;
```

-- cla8_co: cla8 with co from MSB

```
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity cla8_co is

    port (
        a, b : in  std_logic_vector(7 downto 0);
        ci : in  std_logic;
        co : out std_logic;
        s : out std_logic_vector(7 downto 0);
        g, p : out std_logic);

end entity cla8_co;
architecture stdcell of cla8_co is
    signal gh, ph, gl, pl : std_logic;
    signal ch, cl : std_logic;
begin -- architecture cla_stdcell
    adder_L: entity work.cla4(stdcell)
        port map (
            a => a(3 downto 0),
            b => b(3 downto 0),
            ci => ci,
            s => s(3 downto 0),
            g => gl,
            p => pl);
    adder_H: entity work.cla4_co(stdcell)
        port map (
            a => a(7 downto 4),
            b => b(7 downto 4),
            ci => ch,
            co => co,
            s => s(7 downto 4),
            g => gh,
            p => ph);
    -- generating ch, cl, g, h using gpc
    gpc_0: entity work.gpc(stdcell)
        port map (
            gh => gh, ph => ph,
            gl => gl, pl => pl,
            ci => ci,
            ghl => g, phl => p,
            ch => ch, cl => cl
        );
end architecture stdcell;
```

-- 16-bit Carry-lookahead Binary Adder

-- Description:
-- an 16-bit carry-lookahead unsigned adder targeting JSIM standard cell

-- cla16: cla16 without co from MSB

```
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity cla16 is

    port (
        a, b : in  std_logic_vector(15 downto 0);
        ci : in  std_logic;
        s : out std_logic_vector(15 downto 0);
        g, p : out std_logic);

end entity cla16;
architecture stdcell of cla16 is
    signal gh, ph, gl, pl : std_logic;
    signal ch, cl : std_logic;
begin -- architecture cla_stdcell
    adder_L: entity work.cla8(stdcell)
        port map (
            a => a(7 downto 0),
            b => b(7 downto 0),
            ci => ci,
            s => s(7 downto 0),
            g => gl,
            p => pl);
    adder_H: entity work.cla8(stdcell)
        port map (
            a => a(15 downto 8),
            b => b(15 downto 8),
            ci => ch,
            s => s(15 downto 8),
            g => gh,
            p => ph);
    -- generating ch, cl, g, h using gpc
    gpc_0: entity work.gpc(stdcell)
        port map (
            gh => gh, ph => ph,
            gl => gl, pl => pl,
            ci => ci,
            ghl => g, phl => p,
            ch => ch, cl => cl
        );
end architecture stdcell;
```

-- cla16_co: cla16 with co from MSB

```
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity cla16_co is

    port (
        a, b : in  std_logic_vector(15 downto 0);
        ci : in  std_logic;
        co : out std_logic;
        s : out std_logic_vector(15 downto 0);
        g, p : out std_logic);

end entity cla16_co;
architecture stdcell of cla16_co is
    signal gh, ph, gl, pl : std_logic;
    signal ch, cl : std_logic;
begin -- architecture cla_stdcell
    adder_L: entity work.cla8(stdcell)
        port map (
            a => a(7 downto 0),
            b => b(7 downto 0),
            ci => ci,
            s => s(7 downto 0),
            g => gl,
            p => pl);
    adder_H: entity work.cla8_co(stdcell)
        port map (
            a => a(15 downto 8),
            b => b(15 downto 8),
            ci => ch,
            co => co,
            s => s(15 downto 8),
            g => gh,
            p => ph);
    -- generating ch, cl, g, h using gpc
    gpc_0: entity work.gpc(stdcell)
        port map (
            gh => gh, ph => ph,
            gl => gl, pl => pl,
            ci => ci,
            ghl => g, phl => p,
            ch => ch, cl => cl
        );
end architecture stdcell;
```

-- 32-bit Carry-lookahead Binary Adder

-- Description:

-- an 32-bit carry-lookahead unsigned adder targeting JSIM standard cell:

-- cla32: cla32 without co from MSB

```
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity cla32 is
```

```
    port (
        a, b : in  std_logic_vector(31 downto 0);
        ci   : in  std_logic;
        s    : out std_logic_vector(31 downto 0);
        g,p  : out std_logic);
```

end entity cla32;

architecture stdcell of cla32 is

signal gh, ph, gl, pl : std_logic;

signal ch, cl : std_logic;

begin -- architecture cla_stdcell

adder_L: entity work.cla16(stdcell)

```
    port map (
        a => a(15 downto 0),
        b => b(15 downto 0),
        ci => ci,
        s => s(15 downto 0),
        g => gl,
        p => pl);
```

adder_H: entity work.cla16(stdcell)

```
    port map (
        a => a(31 downto 16),
        b => b(31 downto 16),
        ci => ch,
        s => s(31 downto 16),
        g => gh,
        p => ph);
```

-- generating ch, cl, g, h using gpc

gpc_0: entity work.gpc(stdcell)

```
    port map (
        gh => gh, ph => ph,
        gl => gl, pl => pl,
        ci => ci,
        ghl => g, phl => p,
        ch => ch, cl => cl
    );
```

end architecture stdcell;

-- cla32_co: cla32 with co from MSB

```
library ieee;
use ieee.std_logic_1164.all;
library stdcell_lib;
use stdcell_lib.stdcell_pkg.all;
entity cla32_co is
```

```
    port (
        a, b : in  std_logic_vector(31 downto 0);
        ci   : in  std_logic;
        co   : out std_logic;
        s    : out std_logic_vector(31 downto 0);
        g,p  : out std_logic);
```

end entity cla32_co;

architecture stdcell of cla32_co is

signal gh, ph, gl, pl : std_logic;

signal ch, cl : std_logic;

begin -- architecture cla_stdcell

adder_L: entity work.cla16(stdcell)

```
    port map (
        a => a(15 downto 0),
        b => b(15 downto 0),
        ci => ci,
        s => s(15 downto 0),
        g => gl,
        p => pl);
```

adder_H: entity work.cla16_co(stdcell)

```
    port map (
        a => a(31 downto 16),
        b => b(31 downto 16),
        ci => ch,
        co => co,
        s => s(31 downto 16),
        g => gh,
        p => ph);
```

-- generating ch, cl, g, h using gpc

gpc_0: entity work.gpc(stdcell)

```
    port map (
        gh => gh, ph => ph,
        gl => gl, pl => pl,
        ci => ci,
        ghl => g, phl => p,
        ch => ch, cl => cl
    );
```

end architecture stdcell;

Verification of Binary Adders by Exhaustive Test (width=4)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
-- packages for managing text files
-- use std.textio.all;
-- use ieee.std_logic_textio.all;

entity binadder_tb is
end entity binadder_tb;

architecture test of binadder_tb is
    -- inputs
    constant width : natural := 4;
    signal a, b : std_logic_vector(width-1 downto 0);
    signal ci : std_logic;
    -- outputs
    signal s_ex : std_logic_vector(width downto 0);
    signal s_1, s_2 : std_logic_vector(width-1 downto 0);
    signal s_3, s_4 : std_logic_vector(3 downto 0);
    signal co_1, co_2, co_3, co_4 : std_logic;

    -- Timing constants
    constant Tclk : time := 10 ns;
    constant SaT : time := 0.01*Tclk;
    -- Test Pattern #
    signal TP : integer := 0;
    signal idx_i, idx_j : integer := 0;

begin -- architecture beh
    binadder_1: entity work.binadder(rtl)
        generic map (
            width => width)
        port map (a => a, b => b, ci => ci,
            s => s_1, co => co_1);

    binadder_2: entity work.rca(stdcell)
        generic map (
            width => width)
        port map (a => a, b => b, ci => ci,
            s => s_2, co => co_2);

    binadder_3: entity work.csa4(stdcell)
        port map (a => a, b => b, ci => ci,
            s => s_3, co => co_3);

    binadder_4: entity work.cla4_co(stdcell)
        port map (a => a, b => b, ci => ci,
            s => s_4, co => co_4, g => open, p => open);

    TP <= idx_i*(2**width)+idx_j+1;
    process
    begin -- process
        for i in 0 to 2**width-1 loop
            for j in 0 to 2**width-1 loop
                idx_i <= i;
                idx_j <= j;
                a <= std_logic_vector(to_unsigned(i,width));
                b <= std_logic_vector(to_unsigned(j,width));
                -- Carry in = 0
                ci <= '0';
                s_ex <= std_logic_vector(to_unsigned(i+j+0,width+1));
                wait for Tclk-SaT;
                -- checking the result of both adders
                assert s_1=s_ex(width-1 downto 0) report "The sum of binadder(rtl) is wrong!" severity warning;
                assert co_1=s_ex(width) report "The co of binadder(rtl) is wrong!" severity warning;
                assert s_2=s_ex(width-1 downto 0) report "The sum of binadder(rca_stdcell) is wrong!" severity warning;
                assert co_2=s_ex(width) report "The co of binadder(rca_stdcell) is wrong!" severity warning;
                assert s_3=s_ex(width-1 downto 0) report "The sum of binadder(csa_stdcell) is wrong!" severity warning;
                assert co_3=s_ex(width) report "The co of binadder(csa_stdcell) is wrong!" severity warning;
                assert s_4=s_ex(width-1 downto 0) report "The sum of binadder(cla_stdcell) is wrong!" severity warning;
                assert co_4=s_ex(width) report "The co of binadder(cla_stdcell) is wrong!" severity warning;
                wait for SaT;
                TP <= (2**width)*i+j+1;
                -- Carry in = 1
                ci <= '1';
                s_ex <= std_logic_vector(to_unsigned(i+j+1,width+1));
                wait for Tclk-SaT;
                -- checking the result of both adders
                assert s_1=s_ex(width-1 downto 0) report "The sum of binadder(rtl) is wrong!" severity warning;
                assert co_1=s_ex(width) report "The co of binadder(rtl) is wrong!" severity warning;
                assert s_2=s_ex(width-1 downto 0) report "The sum of binadder(rca_stdcell) is wrong!" severity warning;
                assert co_2=s_ex(width) report "The co of binadder(rca_stdcell) is wrong!" severity warning;
                assert s_3=s_ex(width-1 downto 0) report "The sum of binadder(csa_stdcell) is wrong!" severity warning;
                assert co_3=s_ex(width) report "The co of binadder(csa_stdcell) is wrong!" severity warning;
                assert s_4=s_ex(width-1 downto 0) report "The sum of binadder(cla_stdcell) is wrong!" severity warning;
                assert co_4=s_ex(width) report "The co of binadder(cla_stdcell) is wrong!" severity warning;
                wait for SaT;
                TP <= (2**width)*i+j+2;
            end loop; -- j
        end loop; -- i
        assert false report "The exhaustive test of 4-bit binary adders is successful" severity note;
        wait;
    end process;
end architecture test;
```

Verification of Binary Adders by Designing a Set of Test Vectors (width=32)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
-- packages for managing text files
use std.textio.all;
use ieee.std_logic_textio.all;

entity binadder32_tb is
end entity binadder32_tb;

architecture test of binadder32_tb is
-- generic constants
constant width : natural := 32;
-- inputs
signal ci : std_logic;
signal a, b : std_logic_vector(width-1 downto 0);
-- outputs
signal s_ex : std_logic_vector(width-1 downto 0);
signal co_ex : std_logic;
signal s_1, s_2, s_3, s_4 : std_logic_vector(width-1 downto 0);
signal co_1, co_2, co_3, co_4 : std_logic;
signal g, p : std_logic;
-- Timing constants
constant Tclk : time := 100 ns;
constant SaT : time := 0.01*Tclk;
-- Test Pattern #
signal TP : integer := 0;
--
-- function for reading expected result from the test vector file
function LH_to_01 (a : std_logic_vector)
return std_logic_vector is
variable y : std_logic_vector(a'range);
begin
for j in a'range loop
if a(j)='L' then
y(j) := '0';
elsif a(j)='H' then
y(j) := '1';
else
y(j) := 'U';
end if;
end loop;
return y;
end function LH_to_01;

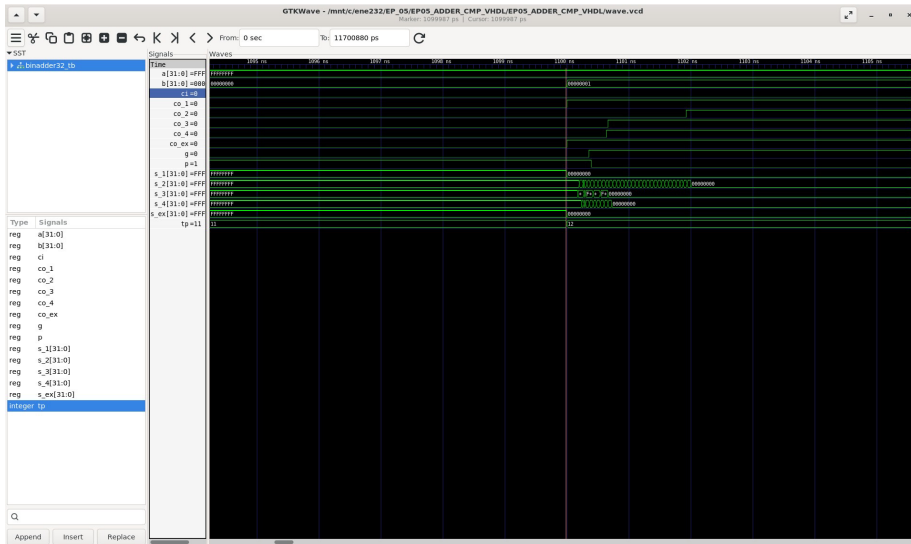
--
-- function for reading expected result from the test vector file
function LH_to_01 (a : std_logic)
return std_logic is
variable y : std_logic;
begin
if a='L' then
y := '0';
elsif a='H' then
y := '1';
else
y := 'U';
end if;
return y;
end function LH_to_01;

begin
binadder32_1: entity work.binadder(rtl)
generic map (
width => width)
port map (
a => a,
b => b,
ci => ci,
co => co_1,
s => s_1);
binadder32_2: entity work.rca(stdcell)
generic map (
width => width)
port map (
a => a,
b => b,
ci => ci,
co => co_2,
s => s_2);
binadder32_3: entity work.csa32(stdcell)
port map (
a => a,
b => b,
ci => ci,
co => co_3,
s => s_3);
binadder32_4: entity work.cla32_co(stdcell)
port map (
a => a,
b => b,
ci => ci,
co => co_4,
s => s_4,
g => g, p => p);

process
file input_file : text open read_mode is "binadder32_testvec.txt";
variable input_line : line;
variable ci_v : std_logic;
variable a_v, b_v : std_logic_vector(31 downto 0);
variable s_exp : std_logic_vector(31 downto 0);
variable co_exp : std_logic;
variable ok : boolean;
begin -- process
while (not endfile(input_file)) loop
TP <= TP+1;
readline(input_file, input_line);
if input_line.all'length=0 or input_line(1)='.' then
next;
end if;
read(input_line, ci_v, ok);
assert ok report "Read ci failed for line" severity failure;
ci <= ci_v;
read(input_line, a_v, ok);
assert ok report "Read a_v failed for line" severity failure;
a <= a_v;
read(input_line, b_v, ok);
assert ok report "Read b_v failed for line" severity failure;
b <= b_v;
read(input_line, co_exp, ok);
assert ok report "Read co_exp failed for line" severity failure;
co_ex <= LH_to_01(co_exp);
read(input_line, s_exp, ok);
assert ok report "Read s_exp failed for line" severity failure;
s_ex <= LH_to_01(s_exp);

-- run for Tclk-SaT, and then check the result with expected value
wait for Tclk-SaT;
-- checking result of binadder32#1 (RTL)
assert s_1=s_ex report "S of RTL design! is wrong!" severity warning;
assert co_1 = co_ex report "co of RTL design is wrong!" severity warning;
-- checking result of binadder32#2 (rca_stdcell)
assert s_2=s_ex report "S of RCA design! is wrong!" severity warning;
assert co_2 = co_ex report "co of RCA design is wrong!" severity warning;
-- checking result of binadder32#3 (csa_stdcell)
assert s_3=s_ex report "S of CSA design! is wrong!" severity warning;
assert co_3 = co_ex report "co of CSA design is wrong!" severity warning;
-- checking result of binadder32#4 (cla_stdcell)
assert s_4=s_ex report "S of CLA design! is wrong!" severity warning;
assert co_4 = co_ex report "co of CLA design is wrong!" severity warning;
-- progressing to the next clock
wait for SaT;
end loop;
assert false report "The function verification of the binadder32 modules is successful" severity note;
wait;
end process;
end architecture test;
```

1. Complete all VHDL models of ripple carry adders (rca), carry-select adders (csa), and carry-lookahead adders (cla) in the respective files, rca_stdcell.vhd, csa_stdcell.vhd, and cla_stdcell.vhd.
2. Use the provided testbench files for verifying your VHDL models of the binary adders. Make corrections as necessary. Record your works for report — this is an individual work.
3. Manually measure t_{PD} of rca32, csa32, and cla32 (you have to find case for each module that produce critical path, and measure
4. Make a comparison of these 3 design concepts of binary adder based on the above testing results of 32-bit adders. Are these data agreed with the expected results from the JSIM simulations you've done in the last exercise problem?



สรุปและอภิปราย

จากที่เราออกแบบและเขียนโค้ด VHDL สำหรับวงจร Binary Adder ทั้ง 3 แบบ ได้แก่ Ripple-Carry Adder (RCA), Carry-select Adder (CSA) และ Carry-Lookahead Adder (CLA) จากที่เราลองทำการวัดเวลาผ่านฟังก์ชัน Testbench และจากการวัดค่า tpd จากหน้าตาของ GTKWave ได้ดังนี้

จากชุด co-2
RCA ได้ค่า tpd = 1940ps

CSA ได้ค่า tpd = 649ps

CLA ได้ค่า tpd = 663ps

เมื่อเปรียบเทียบกันจะเห็นว่า RCA 32 บิตมีความหน่วงเวลาสูงสุด (ช้าที่สุด) เนื่องจากมีโครงสร้างการเชื่อมต่อ Full Adder แบบอนุกรมครบวงในบิตที่สูงสุด

ส่วนความสามารถของบิตจากบิตก่อนหน้าจากครบ 32 บิตทำให้ง่ายเกิดคอมบ์ค่าช้าสะสมตามจำนวนบิตที่เพิ่มขึ้น

ในส่วนของ CSA 32 บิตและ CLA 32 บิตมีความหน่วงเวลาค่อนข้างต่ำ เนื่องจากใช้แนวคิดการประมวลผลแบบขนาน